Transferencia de datos

Son las instrucciones encargadas de mover datos de un sitio a otro de la computadora como pueden ser la memoria, el espacio de E/S y los registros del CPU.

Instrucciones:

- MOV
- XCHG
- IN
- OUT
- PUSH
- POP
- PUSHF
- POPF
- LAHF
- SAHF
- XLAT
- LEA
- LDS
- MOVZX
- MOVSX

XCHG

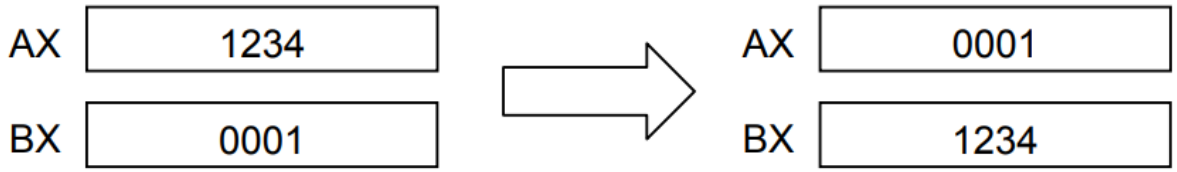
Intercambia el contenido de cualquier registro con el contenido de cualquier otro registro o localidad de memoria, no incluyendo los registros de segmento o intercambios de memoria a memoria.

Ejemplo:

XCHG CL,DH

AX = 472D BX = 1F60 CX = 3184 DX = 0B72

```
XCHG AX, BX
```



IN y OUT

IN:

Lee un dato de un dispositivo de E/S y lo almacena en el registro AL o AX.

OUT:

Transfiere un dato del registro AL o AX a un puerto de E/S.

IN y OUT

Existen dos formas para el direccionamiento de puertos con las instrucciones IN y OUT, las cuales son: **puerto fijo** y **puerto variable**.

- **Puerto fijo:** Permite la transferencia de datos entre AL o AX y un puerto de E/S con dirección de 8 bits.

Ejemplos: `IN AX,85` `IN AL,3F` `OUT 42,AL` `OUT 2A,AX`

- **Puerto variable:** Permite la transferencia de datos entre AL o AX y un puerto de E/S con dirección de 16 bits, y el numero de puerto se almacena en DX.

Ejemplos: `IN AX,DX` `IN AL,DX` `OUT DX,AL` `OUT DX,AX`

IN y OUT

Tabla 2. Instrucciones IN y OUT.

Código de operación	Función
IN AL, pp	Un dato de 8 bits se transfiere del puerto pp a AL
IN AX,pp	Un dato de 16 bits se transfiere del puerto pp a AX
IN AL,DX	Un dato de 8 bits se transfiere del puerto DX a AL
IN AX,DX	Un dato de 16 bits se transfiere del puerto DX a AX
OUT pp,AL	Un dato de 8 bits se transfiere de AL al puerto DX
OUT pp,AX	Un dato de 16 bits se transfiere de AX al puerto pp
OUT DX,AL	Un dato de 16 bits se transfiere de AL al puerto DX
OUT DX,AX	Un dato de 16 bits se transfiere de AX al puerto DX

Nota: pp= un puerto de E/S de con dirección de 8 bits y DX = contiene la dirección de un puerto de E/S con dirección de 16 bits.

PUSH y POP

Se utilizan para empujar o sacar datos del **Segmento de Pila**.

El registro **SP** (Apuntador de Pila) almacena la dirección del segmento de Pila de donde se van a ingresar o sacar datos.

En el 8088 se dice que la Pila crece a direcciones más pequeñas, ya que cuando se **ingresan** datos se **decrementa** el apuntador de pila, y cuando se **extraen** datos se **incrementa** el apuntador. Es decir, conforme se van ingresando datos SP apunta a direcciones cada vez más pequeñas.

PUSH y POP

PUSH:

Ingresa <u>2 bytes</u> de información a la pila.

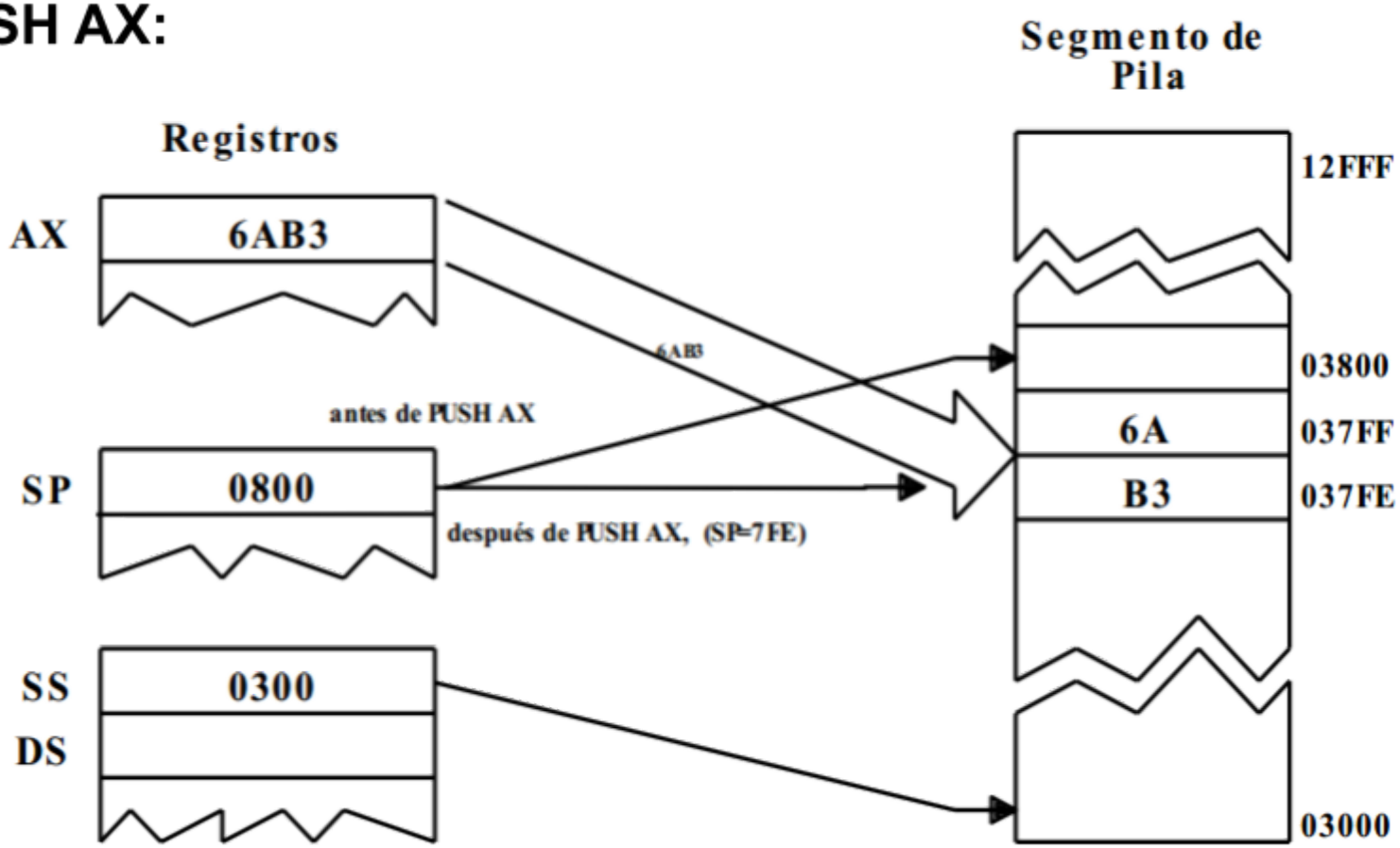
Cuando un dato es metido a la pila, el byte **mas significativo** es almacenado en la localidad direccionada por **SP-1**, y el byte **menos significativo** es almacenado en la localidad **SP-2**.

Después de que el dato se ha empujado a la pila, el registro **SP** es **decrementado en 2**.

PUSH y POP

Ejemplo:

PUSH AX:



PUSH y POP

POP:

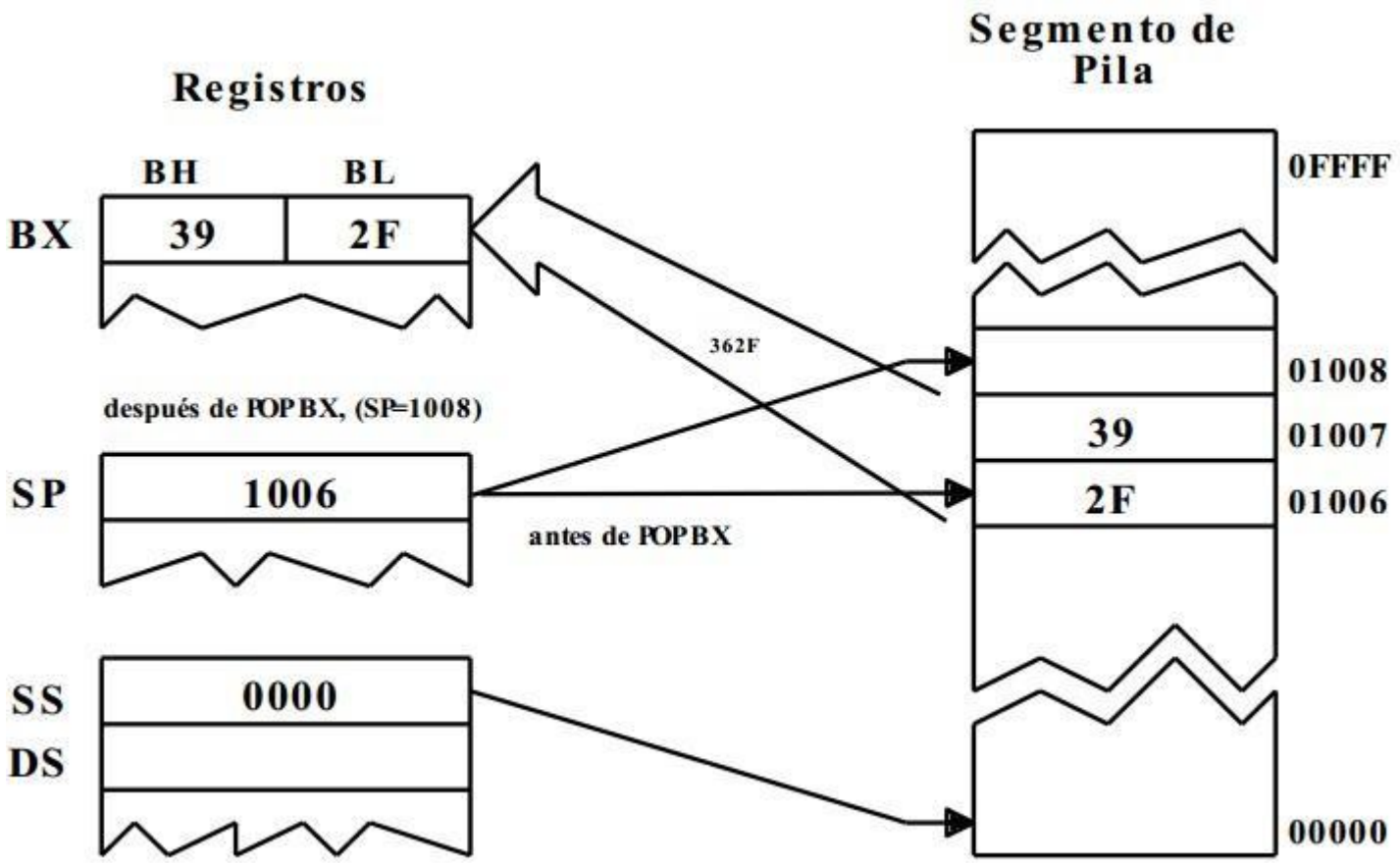
Realiza lo inverso a la instrucción PUSH. POP **remueve <u>2 bytes</u>** de la pila.

Suponga que la instrucción POP BX es ejecutada. El byte apuntado por **SP** es removido de la pila y almacenado en el registro BL. Después se remueve el byte apuntado por **SP+1** y se coloca en el registro BH.

Después de que ambos datos fueron extraídos de la pila, **SP** es **incrementado en 2**.

PUSH y POP

POP BX:



PUSH y POP

Las instrucciones PUSH y POP pueden operar sobre registros, memoria y registros de segmento.

Importante:

Sí se permite empujar a la pila el registro de segmento de Código (CS), pero no se permite extraer un valor de la pila y almacenarlo en CS, ya que no se permite la modificación de este registro.

PUSH CS ✓ (permitido)

POP CS ó **MOV CS,valor** , etc ✖

PUSHF y POPF

PUSHF:

Copia el contenido del registro de banderas a la pila.

POPF:

Realiza la operación inversa de PUSHF, remueve de la pila un dato de 16 bits que es cargado como el contenido del registro de banderas.

LAHF

Copia el byte menos significativo del registro de banderas en el registro AH.

Después de la ejecución de esta instrucción, los bits 7,6,4,2 y 0 de AH son iguales a las banderas S, Z, A, P y C respectivamente.

Registro de banderas:

15							8	7							0
				O	D	I	T	S	Z		A		P		C

SAHF

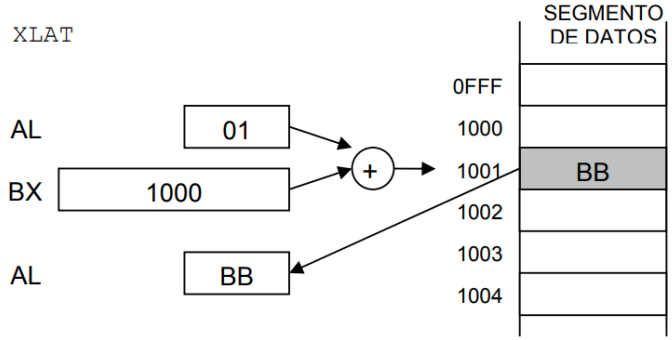
Copia el contenido de AH en el byte menos significativo del registro de banderas.

Después de la ejecución de esta instrucción, las banderas S, Z, A, P y C son iguales a los bits 7, 6, 4, 2 y 0 de AH, respectivamente

XLAT

Se utiliza para realizar una técnica directa de conversión de un código a otro, por medio de una *lookup table*.

La instrucción **XLAT** primero **suma** el contenido de **AL** con el contenido del registro **BX** para formar una dirección del segmento de datos, luego el dato almacenado en esta dirección es cargado en el registro **AL**. El registro BX almacenaría la dirección de inicio de la tabla y AL almacenaría la posición del dato que se quiere cargar en este registro. Esta instrucción no posee operando, ya que siempre opera sobre AL.



LEA

Se utiliza para cargar un registro con la dirección de un dato especificado por un operando (variable).
Ejemplo:

LEA AX,DATA
AX se carga con la dirección de DATA (16 bits), cabe notar que se almacena la dirección y no el contenido de la variable.

LEA BX,[DATA] carga la dirección de DATA en el registro BX.

LEA BX,[DI] carga la dirección apuntada por DI en el registro BX. Es decir, carga el contenido de DI en BX.

MOV BX,[DI] carga el contenido de la localidad direccionada en memoria por DI en el registro BX.

LDS y LES

Estas instrucciones cambian el rango del segmento de Datos o del segmento Extra, y permiten cargar una nueva dirección efectiva.

Las instrucciones **LDS** y **LES** cargan **un registro de 16 bits** con un dato que representaría una dirección (un desplazamiento), y cargan el **registro de segmento** DS o ES con una nueva dirección de segmento.

Estas instrucciones utilizan cualquier modo de direccionamiento a memoria válido para seleccionar la localidad del nuevo desplazamiento y nuevo valor de segmento.

LDS y LES

En el diagrama se muestra la operación de la instrucción **LDS BX,[DI]**, la cual transfiere el contenido (2 bytes) de la localidad de memoria apuntada por DI, al registro BX; y transfiere los siguientes 2 bytes al registro de segmento DS.

Es decir, transfiere la dirección de 32 bits que está almacenada en memoria y es apuntada por DI, a los registro BX y DS

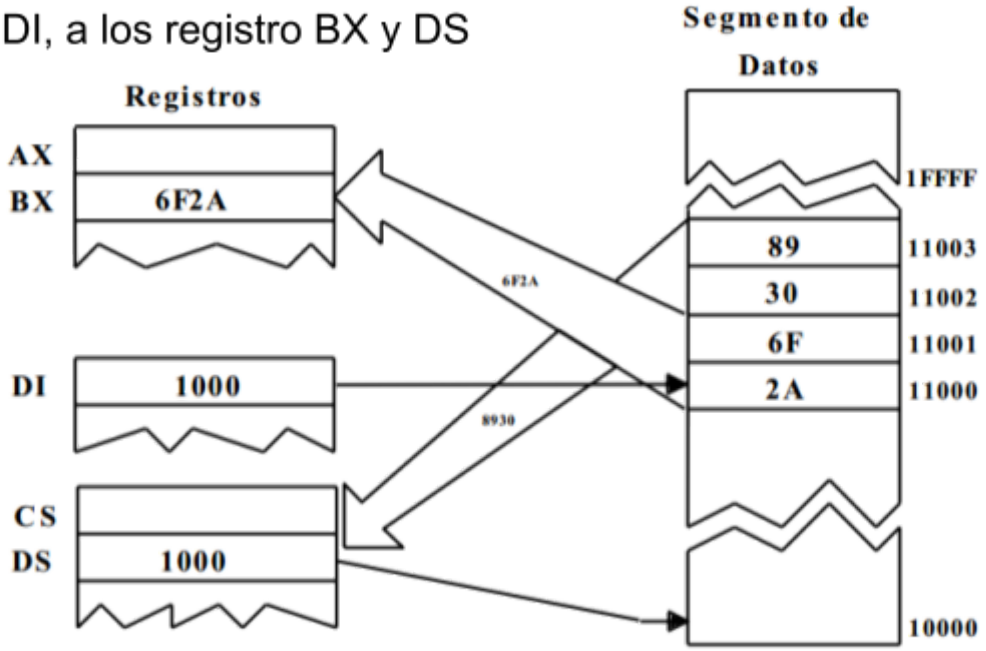


Figura 3. LDS BX,[DI] carga el registro BX con el contenido de la localidad 11000H y 11001H y el registro DS con el contenido de la localidad 11002H y 11003H. Esta instrucción cambia el rango de segmento 1000H-1FFFFH a 89300H-992FFH.

Ejemplos de LEA, LDS y LES

Tabla 4. Instrucción XCHG

Código de operación	Función
LEA AX,DATA	AX se carga con la dirección de DATA (16 bits)
LDS DI,LIST	DI y DS se cargan con la dirección de LIST (32 bits- DS:DI)
LES BX,CAT	BX y ES se cargan con la dirección de CAT (32 bits -ES:BX)

MOVZX

La instrucción MOVZX (mover con extensión de ceros) copia el contenido de un operando de origen a un operando de destino, y extiende con ceros el valor hasta 16 o 32 bits. Esta instrucción se utiliza sólo con enteros sin signo.

```
mov     bx,0A69Bh
movzx   eax,bx                          ; EAX = 0000A69Bh
movzx   edx,bl                          ; EDX = 0000009Bh
movzx   cx,bl                           ; CX = 009Bh
```

```
section .data
byte1 db 9Bh
palab1 dd 0A69Bh

movzx eax, word [palab1]        ;eax = 0000A69Bh
movzx edx, byte [byte1]         ;edx = 0000009Bh
movzx edx, word [byte1]         ;edx = 00009B9Bh
```

MOVSX

La instrucción MOVSX (mover con extensión de signo) copia el contenido de un operando de origen a un operando de destino, y extiende con signo el valor hasta 16 o 32 bits. Esta instrucción se utiliza sólo con enteros con signo.

Para extender con signo un operando, se toma el bit más alto del operando más pequeño y se repite (replica) este bit a lo largo de los bits extendidos en el operando de destino. Suponga que el valor de 8 bits 10001111b se mueve a un destino de 16 bits.

```
mov     bx,0A69Bh
movsx   eax,bx                  ; EAX = FFFFA69Bh
movsx   edx,bl                  ; EDX = FFFFFF9Bh
movsx   cx,bl                   ; CX  = FF9Bh
```